SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH

CUSTOMER SUPPORT DESK
AIM23SLU03 NOSQL DATABASE WITH MONGODB LABORATORY

PROJECT REPORT

Submitted by

Vimochini - E0125103
Priyanka G - E0125099

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Sri Ramachandra Faculty of Engineering and Technology
Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai 600116

OCTOBER, 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH

BONAFIDE CERTIFICATE

Certified that this project report "Customer Support Desk Using MongoDB" is the Bonafide record of work done by Vimochini (E0125103) and Priyanka G (E0125099) who carried out the project under my supervision.

Signature of the Supervisor

Ms. Santhiya C

Assistant Professor,
Department of Medical Engineering
Sri Ramachandra Faculty of Engineering and Technology,
SRIHER, Porur, Chennai 600 116.

Evaluation Date:

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	
1	INTRODUCTION	1
2	LITERATURE REVIEW	3
3	METHODOLOGY	5
4	IMPLEMENTATION	9
5	RESULTS AND DISCUSSIONS	12
6	APPENDICES	
	Appendix 1: Code, Technical Detail	15
	Appendix 2: Screenshots	17
	REFERENCES	19

ABSTRACT

This is a simple support desk, the kind of tool a small business uses to keep track of problems that customers report. Each problem is saved as a ticket. A ticket has a number, a short subject, a description, a status such as Open or Resolved, a priority, the customer who raised it, and the messages exchanged about it. The support agent works on one web page: adding customers, opening tickets, updating them and replying to customers.

Many small teams keep this information in spreadsheets, notes or email threads. Over time, tickets get lost, a status is typed wrongly, the same customer is entered twice and nobody can say how many problems are still open. The information also needs protection, because a mistake made by any tool or person can quietly spoil the records.

The data is stored in MongoDB, a NoSQL document database. MongoDB keeps each ticket and its whole conversation together in one record, and it checks its own rules before anything is saved: an email must look like an email, a status must be one of five allowed words, and only one agent may exist. A short Python program acts as the brain between the web page and the database. It passes requests along, gives each ticket its number and turns any refusal from the database into a clear message. Our tests showed that MongoDB refused every invalid entry, even when we tried to save it directly without going through the page.

CHAPTER 1
INTRODUCTION

1.1. OVERVIEW

Every business that serves customers receives questions and complaints. A customer may be charged twice, may be unable to log in, or may simply want to upgrade a plan. Someone has to write the problem down, decide how urgent it is, reply to the customer, follow it up and finally mark it as solved. This work is called customer support, and the record of each problem is called a ticket.

In this project we built a small support desk for one support agent and the customers that agent serves. The agent uses a single web page to add customers, create tickets, change the status and priority of a ticket, write replies and note down messages that a customer sent by phone or email. All of this information is stored in MongoDB, and the database itself makes sure that the information stays correct.

The project was built for the NoSQL Database with MongoDB laboratory, so the main focus is the database: how the data is organised, what rules it follows and how MongoDB enforces those rules. The web page and the Python program are kept small on purpose, so that the role of the database stays clear.

1.2. PROBLEM STATEMENT

A small support team usually starts with a spreadsheet. It works for the first few tickets, but problems appear quickly. A status can be typed as “Resolvd” instead of “Resolved”. Two rows can end up with the same ticket number. The same customer can be added twice with the same email. The conversation about a ticket does not fit neatly in one cell, so it is scattered across notes and inboxes. Nobody can quickly answer a simple question such as “how many tickets are still open?”

Checking for these mistakes only on the screen does not solve the problem. If the rules live only in the web page, then any other tool that writes to the same data, or any person editing the data directly, can break them without anyone noticing. The rules need to live in the place where the data lives.

The problem we set out to solve is therefore twofold. First, the agent needs one simple place to record and follow customer problems. Second, the stored information must stay correct no matter how it is written, which means the database itself must check it.

1.3. OBJECTIVES

The main objective of this project is to build a working support desk on MongoDB in which the database stores the data and also protects it. The specific objectives are as follows:

- To design a MongoDB database that stores customers, tickets and the messages written on each ticket.
- To make MongoDB check the important rules by itself, such as a valid email, an allowed status and a single agent.
- To prepare ready made views in MongoDB, so the web page can show the ticket list and the totals without extra work.
- To build one simple web page through which the agent can add customers, create tickets, update them and reply.
- To keep the Python program small, letting MongoDB do the storing, checking, joining and counting.
- To test every rule, both through the web page and directly in the database, and record the results.

CHAPTER 2
LITERATURE REVIEW

Before designing the system, we read about how databases have changed and why document databases such as MongoDB suit records like support tickets. This chapter summarises what we learned and how it shaped our design.

2.1 Relational and NoSQL Databases

For many years, most applications stored their data in relational databases. These keep data in tables made of rows and columns, much like a set of linked spreadsheets. Related information is split across tables and joined back together when it is needed. This approach is reliable, but a single business object, such as a ticket with its replies, has to be rebuilt from several tables each time it is read.

NoSQL databases appeared to meet different needs. Cattell surveyed these systems and explained that many of them give up some features of traditional databases, such as transactions across the whole database, in return for easier growth across many servers [1]. Stonebraker, one of the best known database researchers, examined the performance claims made for NoSQL and found several of them unconvincing; he also described document stores, such as MongoDB, as one of the two main groups of NoSQL systems [4]. Leavitt reported that NoSQL databases still had to overcome several technical and market hurdles before they could be widely adopted [2]. Together these papers taught us that NoSQL is not automatically better; it is a different set of trade offs.

2.2 Families of NoSQL Databases

NoSQL databases are usually grouped by the way they store data. Han and colleagues give an overview of NoSQL databases and describe the main families, such as key and value stores, column stores and document stores [3]. Each family suits a different kind of data. A document store keeps each record as a self contained document that can hold lists and nested details. A support ticket, which carries its own list of replies, fits this model naturally. This is the main reason we chose a document database for our project.

2.3 MongoDB as a Document Database

MongoDB stores data as documents that look much like JSON, the format used widely on the web. Similar documents are grouped into collections, and collections are grouped into a database. Chodorow, who worked on MongoDB itself, describes how MongoDB is used both for building applications and for running them [5]. Three features described in her book and in the official MongoDB documentation were especially important for us:

- Documents can contain lists, so all replies on a ticket can live inside the ticket itself.
- Collections can be given rules, called validation rules, which MongoDB checks before saving anything.
- Views can present data in a ready made form, for example tickets together with their customer names.

2.4 Keeping the Rules Inside the Database

MongoDB lets a developer attach a set of rules to a collection using a feature called schema validation. A rule can say that a field must exist, that it must be text or a number, that it must match a pattern such as an email address, or that it must be one of a fixed list of words. When a new or changed record breaks a rule, MongoDB refuses to save it and explains which rule failed.

MongoDB can also refuse duplicates through unique indexes. A special kind called a partial index applies only to records that match a condition. We used this to make sure that only one record can have the role of agent, while any number of customers can be added. These features moved the most important checks into the database, which is exactly what our problem statement required.

2.5 Noticing Changes as They Happen

Some applications need to notice changes in the database straight away. MongoDB offers change streams for this, but they only work when MongoDB runs as a replica set, a group of servers that keep copies of the same data. Our laboratory setup uses a single MongoDB server, so we could not use change streams. Instead, our program simply looks at the database once every second. For a desk with one agent this is fast enough, and we mention it again under challenges in Chapter 5.

CHAPTER 3
METHODOLOGY

This chapter explains how we planned and built the support desk. We followed a simple rule throughout the work: decide what the data must look like, write that rule into MongoDB first, and only then build the screens that use it.

3.1 Workflow Diagram

Figure 3.1 shows what happens each time the agent saves something on the web page. The most important step is the decision in the middle, where MongoDB checks the data against its rules.

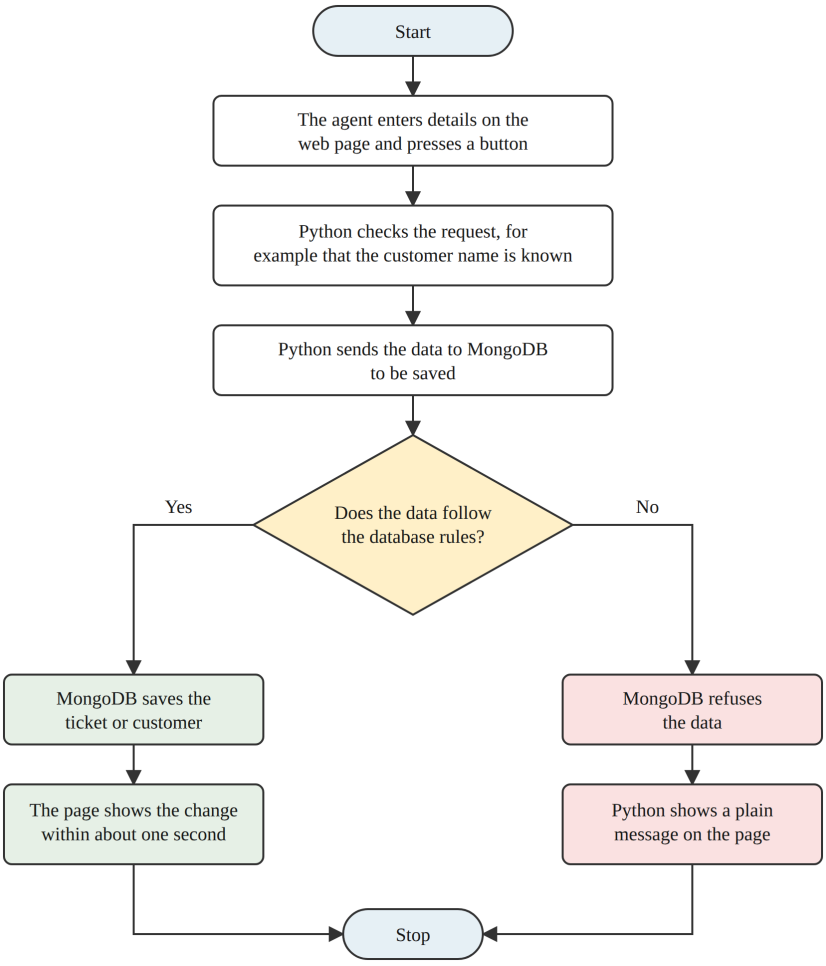


Figure 3.1 Workflow of the support desk

3.2 Modules

The development was carried out in small, separate steps, so that each part could be checked before moving on. Each module below is one stage of that journey, from deciding the goals to the final working page.

3.2.1 Objective Formulation

We began by listing what the desk must do and what the data must never do. The goals were kept practical and simple:

- Store customers and their tickets in one reliable place.
- Keep the conversation on each ticket together with the ticket.
- Refuse wrong or duplicate data in the database itself.
- Give the agent one clear page that anyone can learn in a few minutes.

3.2.2 Proof of Concept

Before writing any screens, we tested the idea directly in MongoDB using mongosh, the MongoDB command line tool. We added a few customers and tickets by hand and then deliberately tried to save wrong data: a ticket with the status “Lost”, a customer with the email “nope” and a second agent. MongoDB refused each of these. This showed us early that the database could carry the rules on its own.

3.2.3 Database Design

We kept the database as small as the problem allowed. It has two collections that hold the data and two views that present it.

Name	Kind	What it holds
users	Collection	Customers and the one support agent: first name, last name, email and role.
tickets	Collection	Each ticket with its number, subject, description, status, priority, customer and list of replies.
ticket_board	View	Tickets joined with the customer’s full name, sorted by ticket number, ready for the page.

Name	Kind	What it holds
ticket_counts	View	The number of tickets in each status, used for the totals at the top of the page.

Table 3.1 Collections and views in the support_system database

A key decision was where to keep the replies. We could have stored them in a separate collection and linked them to tickets, but replies always belong to exactly one ticket and are usually short. Keeping them inside the ticket means a single read brings back the whole conversation, and adding a reply changes only one record.

3.2.4 Application Design

Around the database we placed two small pieces. The Python program, which we call the brain, receives requests from the page, checks the few things the database cannot check and passes everything else to MongoDB. The web page shows the data and collects the agent's input. Figure 4.1 in the next chapter shows how these parts connect.

3.2.5 Working Model / Dashboard / Application

The final system works as one unit:

- MongoDB stores every customer, ticket and reply, and refuses any data that breaks its rules.
- The Python program passes requests to MongoDB, numbers new tickets and turns refusals into friendly messages.
- The web page displays three totals (all tickets, open tickets and resolved tickets), a form for new tickets, a form for new customers and the list of tickets.
- Each ticket on the page can have its status and priority changed, and the agent can add a reply or note a message from the customer.

3.3 MongoDB Implementation

Database: All data lives in one MongoDB database named support_system. It is created by a single setup file, support_system.mongodb.js, which can be run in mongosh or in the MongoDB extension for VS Code. The file can be run again at any time without harming existing data.

Rules: Each collection carries validation rules. Table 3.2 lists the rules and how MongoDB enforces each one.

Rule	How MongoDB enforces it
Status must be Open, In Progress, Pending, Resolved or Closed	Validation rule with a fixed list of allowed words
Priority must be Low, Medium, High or Critical, with a score from 1 to 5	Validation rule with a fixed list and a number range
Email must look like name@domain	Validation rule with an email pattern
No two people may share an email	Unique index on email
There can be only one agent	Partial unique index on the role of agent
No two tickets may share a number	Unique index on ticket number
Names, subject, description and replies cannot be empty	Validation rule requiring at least one character

Table 3.2 Rules kept by MongoDB

Views: Two views do the reading work. The ticket_board view looks up each ticket's customer, joins in the customer's full name and sorts the tickets by number. The ticket_counts view groups tickets by status and counts them. The page never has to combine or count the data itself.

The agent record: Because the desk has one agent, the setup file creates that agent record automatically if it does not exist yet. The agent's name is set in the file before the first run, and later runs leave it unchanged.

Why MongoDB is central to this design

MongoDB is the only place where data is kept. Restarting the program or the browser loses nothing.

MongoDB checks every rule for every writer, whether the data comes from our page, another program or a person typing commands.

MongoDB prepares the ticket list and the totals through views, so the rest of the system stays small and easy to read.

CHAPTER 4
IMPLEMENTATION

This chapter describes how the system was actually built and connected. It covers the database, the Python program and the web page, and then how the three were brought together.

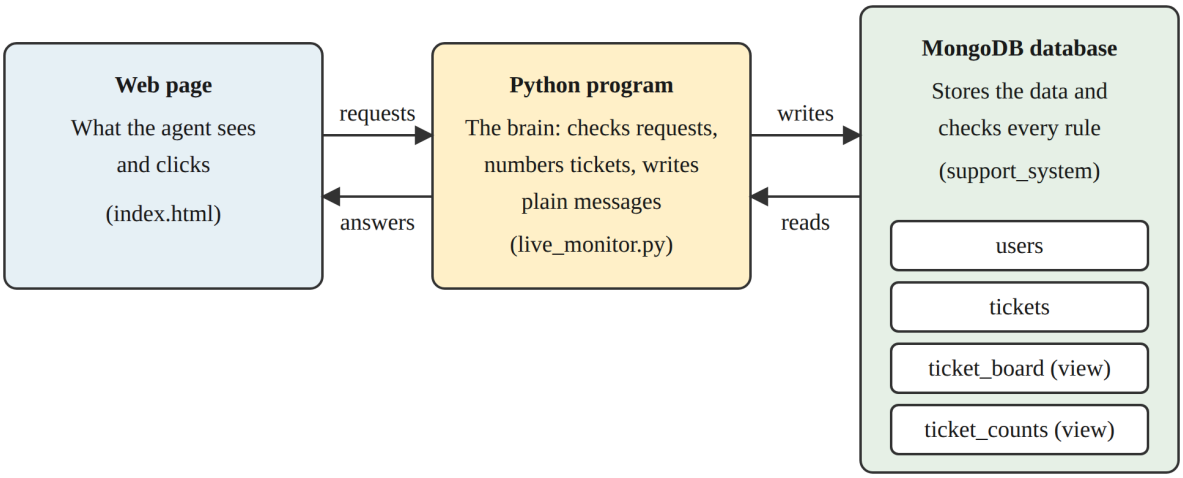


Figure 4.1 How the three parts of the system connect

4.1 Database Implementation

The database was built with the following tools and parts:

- MongoDB server: runs on the same computer and stores the support_system database.
- mongosh and the VS Code MongoDB extension: used to run the setup file and to test the rules directly.
- The setup file support_system.mongodb.js: creates the collections and their rules, the three unique indexes, the agent record and the two views.

The setup file works through five steps in order. It selects the database, creates the users and tickets collections if they are missing, applies the validation rules to them, creates the unique indexes, adds the agent record if needed, and finally rebuilds the two views so they always match the file. The file adds no customers or tickets; those are entered by the agent through the page.

4.2 Application Logic (Python)

The Python program, live_monitor.py, connects to MongoDB using PyMongo, the official MongoDB driver for Python. It uses only Python's built in web server, so no extra framework is needed. Its key responsibilities are:

- Receiving each request from the page and sending it to the right place in MongoDB.
- Finding the customer whose full name was typed in the ticket form, ignoring capital letters and extra spaces, and reporting clearly if the name is unknown or shared by two customers.
- Giving each new ticket the next number, which is the highest existing number plus one.
- Turning a refusal from MongoDB into a plain message. For example, when MongoDB rejects a status, the page shows "Pick a status from the list."
- Reading the totals and the list of people from MongoDB once every second, so the page stays current.

The program answers seven kinds of request from the page, listed in Table 4.1.

Request	What it does	Where the data comes from
Open the page	Sends the web page to the browser	The file index.html
Get totals	Returns totals, people and allowed statuses	users collection and ticket_counts view
Get tickets	Returns the ticket list, optionally for one status	ticket_board view
Add customer	Saves a new customer	users collection
Create ticket	Saves a new ticket	tickets collection
Update ticket	Changes the status or priority	tickets collection
Add reply	Adds a reply from the agent or a message from the customer	tickets collection

Table 4.1 Requests handled by the Python program

4.3 Software Implementation (Web Page)

The web page, index.html, is a single file that runs in any modern browser. It was designed so that a person with no technical background can use it after a short look. From top to bottom it contains:

- Three boxes showing the total number of tickets, the tickets marked Open and the tickets marked Resolved.
- A New ticket form with boxes for the subject, the customer’s full name, the priority and a description.
- A Customers form with boxes for first name, last name and email, and a line listing the customers already added.
- The list of tickets, with a menu to show only tickets of one status.
- For each ticket: menus to change its status and priority, the replies so far, a box to type a new reply, a Reply button for the agent and a Customer message button for noting what the customer said.

The totals refresh every second. The ticket list refreshes after every action and every fifteen seconds, and anything the agent is typing in a reply box is kept during the refresh.

4.4 Integration Plan

The system was built and tested in four stages:

1. Database first: the setup file was written and the rules were tested directly in mongosh.
2. Python program: the requests were added one at a time and each was tested against the database.
3. Web page: the forms and the ticket list were connected to the Python program.
4. Full testing: every rule was tested again through the page, and the sample data was entered and removed.

To run the finished system, MongoDB is started first, the setup file is run once, PyMongo is installed with the command pip install pymongo, the program is started with python scripts/live_monitor.py and the page is opened at http://127.0.0.1:8765 in a browser.

CHAPTER 5
RESULTS AND DISCUSSIONS

This chapter presents what we observed while using and testing the support desk, what we learned, the difficulties we faced and how the system could grow in the future.

5.1 Observations

The support desk worked as intended. The key observations were:

- MongoDB refused every invalid entry we tried, both through the page and when we wrote directly to the database.
- Every refusal reached the agent as a short, plain message rather than a technical error.
- Ticket numbers stayed in order with no gaps, even after several rejected attempts.
- Changes appeared on the page within one to two seconds.
- The page stayed simple to use with several tickets and several replies on each.

5.2 Key Findings

- Placing the rules inside MongoDB protected the data far better than checks on the screen alone, because the rules applied to every writer.
- Storing replies inside each ticket made both saving and showing a conversation simple.
- MongoDB views removed the need for the program to join or count data, which kept the Python program short.
- A partial unique index was a neat way to allow only one agent while allowing many customers.

5.3 Challenges Faced

- No live updates from the database: change streams need a replica set, which our single server did not have, so the program checks the database every second instead.

- Turning database errors into messages: MongoDB reports which rule failed in a technical form, so the program looks for the rule name in the error and picks the matching message. We tested this for every rule.
- Gaps in ticket numbers: our first version used a separate counter, which skipped a number every time a ticket was rejected. We replaced it with “highest number plus one”, which keeps the numbers in order. The only side effect is that if the newest ticket were deleted, its number would be used again; the page has no delete button, so this does not happen in normal use.
- Keeping the project focused: early versions included teams, several agents, reports and a history log. We removed these to keep the system small and the role of the database clear.

5.4 Future Enhancements

- A login for the agent, so that only the agent can change tickets.
- Running MongoDB as a replica set, so the page can show changes instantly using change streams and keep a history of changes.
- Search by customer name or ticket subject.
- Support for more than one agent, with each ticket assigned to an agent.
- Regular backups of the database.

5.5 Test Results

We tested every rule in two ways: through the same requests the web page sends, and directly in mongosh, skipping the Python program completely. All tests passed.

Test	What we tried	What happened	Result
Repeated email	A second customer with an email already in use	“That email is already used.”	Pass
Wrong email	The email “nope”	“Enter a valid email.”	Pass
Missing name	A blank first name	“First and last name are needed.”	Pass
Missing subject	A blank subject	“Subject and description are needed.”	Pass

Test	What we tried	What happened	Result
Wrong priority	The priority “Huge”	“Pick a priority from the list.”	Pass
Unknown customer	The customer name “Nobody”	“No customer called “Nobody”. Check the spelling.”	Pass
Empty reply	A blank reply	“Write a comment first.”	Pass
Wrong status	The status “Lost”	“Pick a status from the list.”	Pass
Ticket numbers	Two rejected tickets, then two valid ones	Numbered 1 and 2, no gap	Pass
Replies	A reply from the agent and a message from the customer	Both shown with the writer’s name	Pass
Status filter	Show only Pending tickets	Only Pending tickets listed	Pass

Table 5.1 Tests through the web page requests

Test written directly in mongosh	MongoDB’s answer	Result
Ticket with the status “Lost”	Document failed validation	Refused
Customer with the email “nope”	Document failed validation	Refused
A second agent	Duplicate key error on the role index	Refused

Table 5.2 Tests made directly in the database

What the results say about MongoDB

The direct tests in Table 5.2 are the strongest evidence in this report. They show that the data is protected by MongoDB itself, not by our page or our program.

Because the rules live in one setup file, anyone can read exactly what the data is allowed to look like without reading any application code.

5.6 Conclusion

We set out to build a support desk where the database does more than store data. The finished system shows that MongoDB can store every ticket with its full conversation, check all the

important rules by itself and prepare the data the page needs. The Python program stayed short because MongoDB did most of the work, and the page stayed simple enough for anyone to use.

The project also showed us the limits of our setup. A single MongoDB server cannot announce changes, the desk has only one agent and the page has no login. These limits are listed above together with the steps that would remove them. Overall, the project gave us a clear, practical understanding of how a NoSQL document database can be the foundation of a real application.

APPENDICES

APPENDIX 1: CODE

Excerpt 1: rules for the tickets collection, from support_system.mongodb.js

```
db.runCommand({ collMod: "tickets", validator: { $jsonSchema: {
  bsonType: "object",
  required: ["ticket_no", "subject", "description", "status", "customer_id"],
  properties: {
    status: { enum: ["Open", "In Progress", "Pending", "Resolved", "Closed"] },
    priority: { bsonType: "object", properties: {
      level: { enum: ["Low", "Medium", "High", "Critical"] },
      score: { bsonType: "number", minimum: 1, maximum: 5 } } }
  } } } })
```

Excerpt 2: the index that allows only one agent, from support_system.mongodb.js

```
db.users.createIndex({ email: 1 }, { unique: true });
db.users.createIndex({ role: 1 },
  { unique: true, partialFilterExpression: { role: "agent" } });
```

Excerpt 3: the view that prepares the ticket list, from support_system.mongodb.js

```
db.createView("ticket_board", "tickets", [
  { $lookup: { from: "users", localField: "customer_id",
               foreignField: "_id", as: "customer" } },
  { $unwind: "$customer" },
  { $project: { ticket_no: 1, subject: 1, description: 1, status: 1, comments: 1,
      priority: "$priority.level",
      customer: { $concat: ["$customer.firstName", " ", "$customer.lastName"] } } },
  { $sort: { ticket_no: 1 } }
]);
```

Excerpt 4: adding a reply to a ticket, from live_monitor.py

```
def add_comment(d):
    _id = oid(d.get("id"))
    ticket = db.tickets.find_one({"_id": _id})
    if not ticket:
```

```
        raise ValueError("That ticket no longer exists.")
    if d.get("side") not in ("customer", "agent"):
        raise ValueError("Pick who is writing.")
    author = ticket["customer_id"] if d["side"] == "customer" else agent_id()
    db.tickets.update_one({"_id": _id}, {"$push": {"comments": {
        "author_id": author,
        "message": (d.get("message") or "").strip(),
        "timestamp": datetime.now(timezone.utc)}}})
```

Excerpt 5: turning a MongoDB refusal into a plain message, from live_monitor.py

```
def explain(exc):
    if isinstance(exc, DuplicateKeyError):
        if "email_1" in str(exc):
            return "That email is already used."
        return "There is already an agent. This desk has one agent."
    text = str(exc.details)
    for rule, message in RULE_MESSAGES.items():
        if f"'{rule}'" in text:
            return message
    return "Could not save. Please try again."
```

APPENDIX 2: SCREENSHOTS

The screenshots below were taken from the running system with sample customers and tickets. The sample data was removed afterwards. The agent appears under the placeholder name Sam Support, which is set in the setup file.

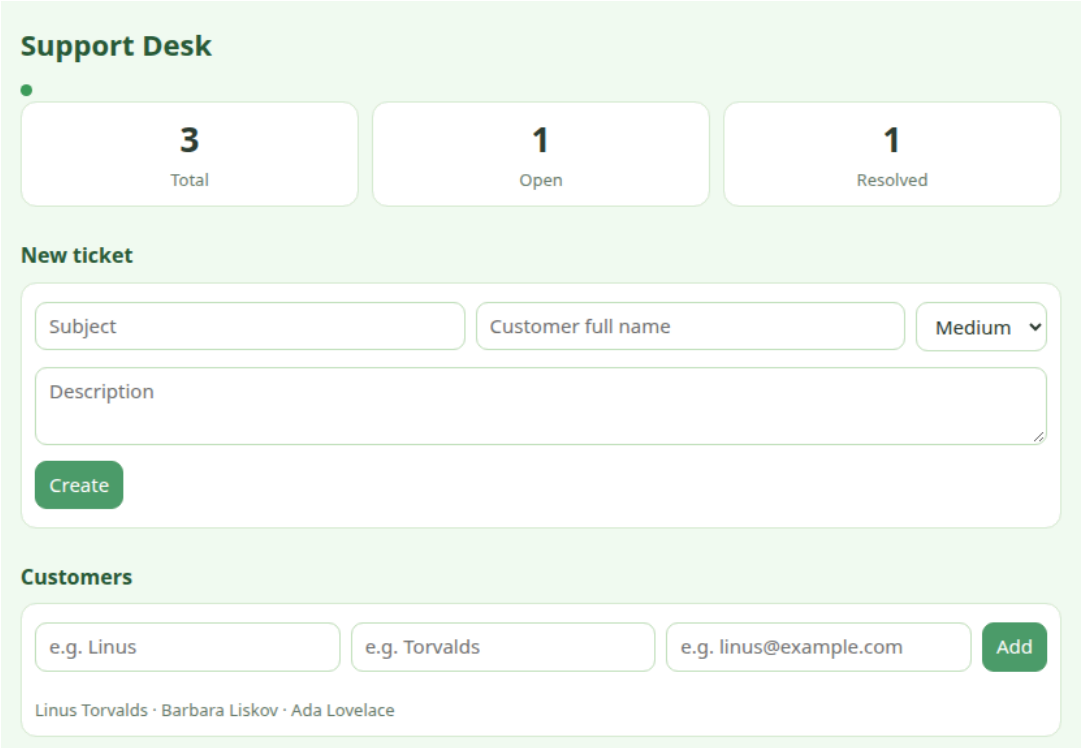


Figure A2.1 The totals, the New ticket form and the Customers form

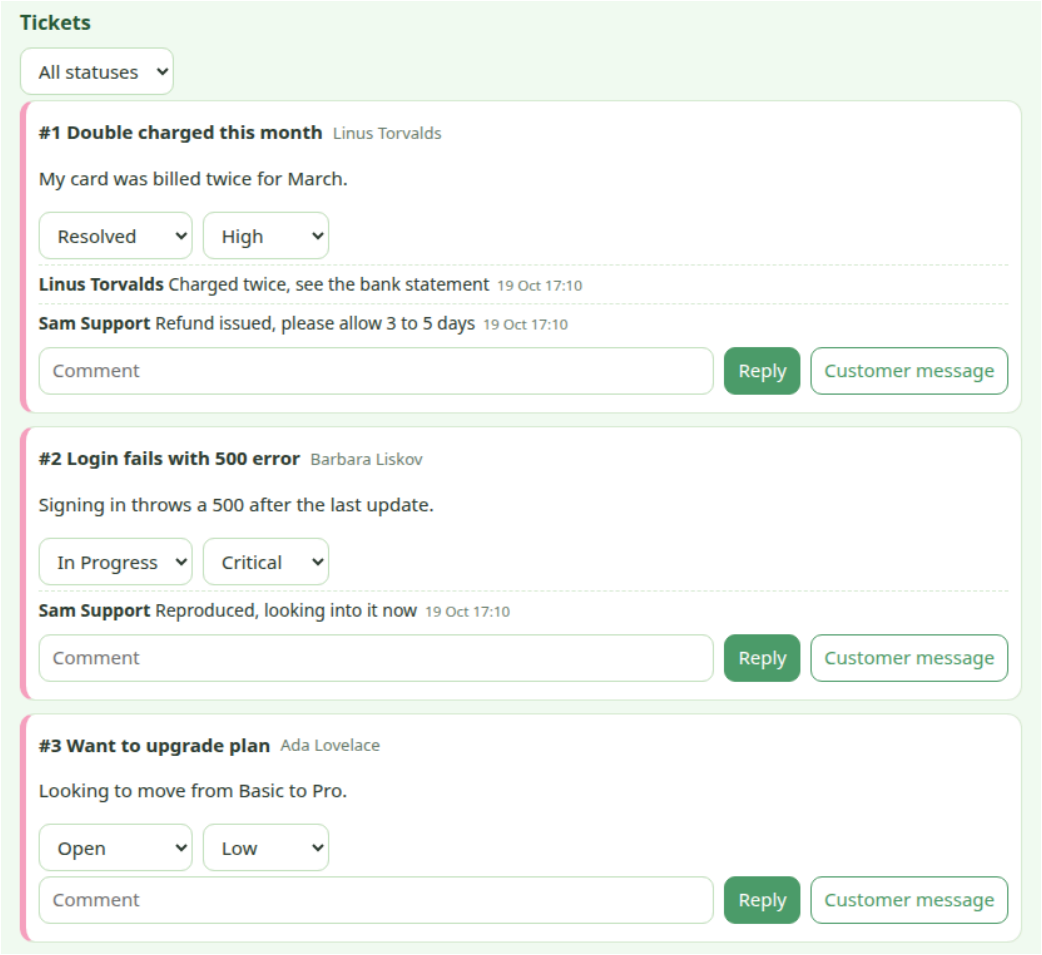


Figure A2.2 The ticket list with status and priority menus and replies

Output from mongosh when invalid data was written directly to the database:

Insert with status Lost: Document failed validation
Insert with email nope: Document failed validation
Insert of a second agent: E11000 duplicate key error collection:
support_system.users index: role_1 dup key: { role: "agent" }

REFERENCES

1. Cattell, R. (2011), “Scalable SQL and NoSQL Data Stores”, ACM SIGMOD Record, 39(4), pp. 12 to 27.
2. Leavitt, N. (2010), “Will NoSQL Databases Live Up to Their Promise?”, Computer, 43(2), pp. 12 to 14.
3. Han, J., Haihong, E., Le, G., & Du, J. (2011), “Survey on NoSQL Database”, Proceedings of the 6th International Conference on Pervasive Computing and Applications, pp. 363 to 366.
4. Stonebraker, M. (2010), “SQL Databases v. NoSQL Databases”, Communications of the ACM, 53(4). DOI 10.1145/1721654.1721659.
5. Chodorow, K. (2013), MongoDB: The Definitive Guide, Second Edition, O’Reilly Media.

Web References

1. MongoDB Manual, MongoDB Inc. https://www.mongodb.com/docs/manual/
2. MongoDB Manual: $jsonSchema (schema validation). https://www.mongodb.com/docs/manual/reference/operator/query/jsonschema/
3. MongoDB Manual: Indexes. https://www.mongodb.com/docs/manual/indexes/
4. MongoDB Manual: Views. https://www.mongodb.com/docs/manual/core/views/
5. MongoDB Manual: Aggregation. https://www.mongodb.com/docs/manual/aggregation/
6. MongoDB Manual: Change Streams. https://www.mongodb.com/docs/manual/changestreams/
7. PyMongo Documentation. https://pymongo.readthedocs.io/en/stable/